SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications.* (BTL 5)

Assessment Tool Weightage: 50%

Duration : 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Design Task

Code of Conduct:

I _____BAIRAGANI DINESH-192372189_____ (Name / Reg No) certify that this submission is my original work and that I have adhered

to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:BAIRAGANI DINESH

Annexure A

Design Task

1. **Hybrid AI Recommendation Engine Design**
   Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.

2. **Smart Disaster Detection System**

Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.

3. **AI-Based Fraud Detection Framework**

Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.

4. **Autonomous Supply Chain Optimization System**

Construct a distributed AI system using PySpark for optimizing supply chain operations

(inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.

5. **Personalized Learning Analytics Platform**

Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

RUBRICS FOR CALCULATION

Evaluation Criteria	**Problem Understanding & Requirement Analysis**	**System Architecture Design**	**Use of PySpark & RDD Operations**	**Intelligent Agent Integration**	**Scalability & Distributed Computing Design**	**Data Flow & Processing Pipeline**	**Innovation & Creativity**	**Clarity, Presentation & Documentation**
Weightage	10%	20%	15%	15%	10%	10%	10%	10%

Criteria	**Excellent (5)**	**Good (4)**	**Average (3)**	**Poor (2)**

Problem Understanding & Requirement Analysis	Clearly defines problem, objectives, and constraints	Mostly clear with minor gaps	Basic understanding	Unclear or incorrect
System Architecture Design	Complete, scalable, well-structured architecture with diagrams	Good design with minor issues	Basic design, lacks detail	Poor/incomplete
Use of PySpark & RDD Operations	Efficient and correct use of RDD transformations/actions	Mostly correct usage	Limited or partial use	Incorrect/missing
Intelligent Agent Integration	Strong integration with clear agent roles and logic	Good integration	Basic integration	Poor/no integration
Scalability & Distributed Computing Design	Clearly addresses scalability, fault tolerance, parallelism	Covers most aspects	Limited consideration	Not addressed
Data Flow & Processing Pipeline	Clear, logical, and well-defined data flow	Mostly clear	Some gaps	Poorly defined
Innovation & Creativity	Highly innovative, realistic, and impactful solution	Some innovation	Basic idea	No originality
Clarity, Presentation & Documentation	Very clear, neat diagrams, well-organized report	Mostly clear	Somewhat organized	Poor presentation

1. Hybrid AI Recommendation Engine Design

Problem Understanding

The objective is to design a scalable recommendation system that combines **content-based filtering** and **collaborative filtering**. The system recommends products, movies, courses, or services based on user preferences and the behavior of similar users.

System Architecture

Data Sources → PySpark RDD Processing → Feature Extraction → Recommendation Models → Intelligent Agents → User Interface

- User-item interaction data is collected from ratings, clicks, searches, purchases, and browsing history.
- Product/item information such as category, keywords, and description is used for content-based filtering.

PySpark RDD Operations

RDDs can process millions of user-item records in parallel.

```
interactions = sc.textFile("user_items.csv")


data = interactions.map(lambda x: x.split(","))


ratings = data.filter(lambda x: float(x[2]) >= 3)


user_items = ratings.map(lambda x: (x[0], x[1]))


item_count = user_items.mapValues(lambda x: 1).reduceByKey(lambda a,b: a+b)


result = item_count.collect()
```

Important RDD operations:

- map() – transforms records.
- filter() – removes unwanted records.
- flatMap() – extracts multiple values.

- reduceByKey() – aggregates similar keys.
- groupByKey() – groups records.
- join() – combines datasets.

Hybrid Recommendation

Scalability

RDD partitions allow data to be processed simultaneously on multiple machines. Spark provides fault tolerance through RDD lineage, allowing lost partitions to be recomputed.

Data Flow

User Activity → Data Collection → RDD Processing → Feature Extraction → Recommendation → Agent Feedback → Updated Recommendation

Innovation

The system can give more importance to recent user behavior, making recommendations dynamically adapt to changing interests.

Conclusion

The proposed hybrid recommendation engine combines the strengths of content-based and collaborative filtering. PySpark provides scalable distributed processing, while intelligent agents provide real-time personalization and continuous improvement.

2. Smart Disaster Detection System

Problem Understanding

The objective is to detect disasters such as **floods and earthquakes** using large-scale data from sensors, satellite images, weather information, and social media.

System Architecture

Sensors + Satellite Images + Social Media + Weather Data
↓
Data Collection Layer
↓
PySpark Distributed Processing
↓
Data Fusion & AI Detection
↓
Intelligent Agents
↓

Early Warning System

↓

Authorities & Public

Data Processing Using PySpark

Large volumes of sensor and social-media data are converted into RDDs.

```
data = sc.textFile("disaster_data.csv")


records = data.map(lambda x: x.split(","))


alerts = records.filter(lambda x: float(x[2]) > 7)


locations = alerts.map(lambda x: (x[0], x[1]))


count = locations.mapValues(lambda x: 1)\
        .reduceByKey(lambda a,b: a+b
```

If several sources indicate the same event, the system increases the disaster confidence score.

Intelligent Agents

- **Sensor Agent:** monitors sensor readings.
- **Satellite Agent:** analyzes satellite information.
- **Social Media Agent:** detects disaster-related reports.

Scalability

PySpark divides large datasets into partitions and processes them across multiple worker nodes. This enables real-time processing of millions of sensor and social-media records.

Data Flow

Data Sources → RDD Creation → Cleaning → Feature Extraction → Data Fusion → AI Detection → Agent Coordination → Warning

Innovation

The system can calculate a **Disaster Confidence Score** using multiple data sources and automatically increase the warning level when confidence becomes high.

Conclusion

The proposed disaster detection system provides scalable and fast disaster monitoring. PySpark handles distributed processing, while intelligent agents coordinate detection, warnings, and emergency response.

3. AI-Based Fraud Detection Framework

Problem Understanding

The objective is to detect fraudulent financial transactions from millions of transactions using distributed AI processing.

System Architecture

Banking Transactions
↓
PySpark RDD Layer
↓
Data Cleaning & Feature Extraction
↓
Distributed Anomaly Detection
↓
Intelligent Fraud Agents
↓
Risk Score
↓
Alert / Block Transaction

PySpark RDD Processing

```
transactions = sc.textFile("transactions.csv")


data = transactions.map(lambda x: x.split(","))


valid = data.filter(lambda x: float(x[2]) > 0)


amounts = valid.map(lambda x: (x[1], float(x[2])))
```

```
total = amounts.reduceByKey(lambda a,b: a+b)


suspicious = valid.filter(lambda x: float(x[2]) > 100000)
```

RDD operations used:

- map() for transformation.
- filter() for suspicious transactions.
- reduceByKey() for aggregation.
- groupByKey() for customer transaction analysis.
- join() for combining customer and transaction data.

Anomaly Detection

Features such as:

- Transaction amount
- Transaction frequency
- Location
- Device
- Time
- Previous transaction history

are analyzed to identify unusual behavior.

A transaction can receive a **Fraud Risk Score**. High-risk transactions are sent to the alert system.

Intelligent Agents

- **Transaction Agent:** monitors transactions.
- **Pattern Agent:** identifies unusual patterns.
- **Risk Agent:** calculates fraud probability.
- **Alert Agent:** generates alerts.
- **Learning Agent:** learns from confirmed fraud and legitimate transactions.

Scalability

PySpark distributes transactions across multiple nodes. RDD fault tolerance allows failed partitions to be recomputed, while parallel processing enables high-volume transaction analysis.

Data Flow

Transaction → RDD → Cleaning → Feature Extraction → Anomaly Detection → Risk Score → Agent Decision → Alert

Innovation

The system can detect new fraud patterns by continuously learning from confirmed fraudulent transactions and user feedback.

Conclusion

The framework provides scalable fraud detection using PySpark RDDs and intelligent agents. Distributed processing reduces processing time and allows financial institutions to monitor large transaction volumes efficiently.

4. Autonomous Supply Chain Optimization System

Problem Understanding

The objective is to optimize inventory, transportation, demand forecasting, and delivery operations using distributed AI and intelligent agents.

System Architecture

Sales + Inventory + Supplier + GPS + Customer Data
↓
PySpark RDD Processing
↓
Demand Forecasting
↓
Inventory & Logistics Optimization
↓
Intelligent Agents
↓
Decentralized Decisions
↓
Optimized Supply Chain

RDD Operations

```
sales = sc.textFile("sales.csv")
```

```
data = sales.map(lambda x: x.split(","))


product_sales = data.map(lambda x: (x[1], int(x[2])))


total_sales = product_sales.reduceByKey(lambda a,b: a+b)


high_demand = total_sales.filter(lambda x: x[1] > 1000)
```

RDD operations include:

- map() – transforms sales records.
- filter() – identifies high-demand products.
- reduceByKey() – calculates total product demand.
- join() – combines inventory and sales information.
- sortByKey() – sorts products according to demand.

Intelligent Agents

- **Demand Agent:** predicts future demand.
- **Inventory Agent:** decides when to reorder.

Decentralized Decision Making

For example, when inventory becomes low:

1. Inventory Agent detects low stock.
2. Demand Agent checks predicted demand.

Scalability

PySpark processes sales, inventory, and logistics data across distributed worker nodes. RDD partitioning allows different regions, products, or warehouses to be processed simultaneously.

Data Flow

Sales Data → RDD Processing → Demand Prediction → Inventory Analysis → Agent Decisions → Logistics Optimization → Final Action

Innovation

The system can automatically adjust inventory and delivery decisions according to real-time demand, traffic, supplier availability, and transportation costs.

Conclusion

The proposed autonomous supply chain system uses PySpark for scalable analytics and intelligent agents for decentralized decision-making. It reduces cost, prevents stock shortages, and improves delivery efficiency.

5. Personalized Learning Analytics Platform

Problem Understanding

The objective is to develop an AI-based learning platform that analyzes student behavior and performance and provides personalized learning content.

System Architecture

Student Activity + Quiz Scores + Attendance + Learning Time
↓
PySpark RDD Processing
↓
Student Performance Analysis
↓
AI Prediction
↓
Intelligent Learning Agents
↓
Personalized Content
↓
Real-Time Feedback

PySpark RDD Operations

```
students = sc.textFile("student_data.csv")


data = students.map(lambda x: x.split(","))


scores = data.map(lambda x: (x[0], float(x[2])))


average = scores.reduceByKey(lambda a,b: a+b)
```

weak_students = scores.filter(lambda x: x[1] < 50)

RDD operations:

- map() – transforms student records.
- filter() – identifies weak-performing students.
- reduceByKey() – calculates performance statistics.
- groupByKey() – groups activities by student.
- join() – combines performance with course information.

Intelligent Agents

- **Student Agent:** monitors individual learning behavior.
- **Performance Agent:** analyzes marks and progress.
- **Content Agent:** selects suitable learning materials.
- **Feedback Agent:** provides immediate feedback.
- **Adaptation Agent:** modifies difficulty based on performance.
- **Teacher Agent:** provides summarized performance information to teachers.

Adaptive Learning

If a student performs poorly in a topic:

1. Performance Agent detects the weakness.
2. Content Agent recommends basic learning material.

Scalability

PySpark RDDs allow thousands of student records to be processed simultaneously across multiple machines. Distributed processing supports large educational platforms with millions of activities.

Data Flow

Student Activity → RDD → Data Cleaning → Performance Analysis → AI Prediction → Agent Decision → Personalized Content → Feedback

Innovation

The system can create an individual **Learning Profile** for every student based on performance, learning speed, topic difficulty, and engagement.

Conclusion

The personalized learning platform combines PySpark's distributed processing with intelligent agents to provide adaptive education. It can analyze thousands of learners efficiently and deliver personalized content and real-time feedback.